

CodeHogwarts — гайд по проекту

Короткая карта: что это, на чём написано, где что лежит и как оно работает. Читается за десять минут, дальше можно возвращаться к нужному разделу.

Живой стенд: <https://maks.my/codehog/>

1. Что это

Тренажёр Python в виде игры. Человек заходит без регистрации, проходит тест на шесть вопросов, получает курс под свой уровень и решает задания прямо в браузере. Код не проверяется по образцу, а **реально исполняется на сервере**. За пройденные уроки начисляются монеты, каждые пять уроков даётся вращение колеса, на монеты покупается одежда для персонажа.

Смысл в одной фразе: **выполненный код превращается в видимые улучшения ежа**.

2. На чём написано

Слой	Технология	Зачем именно она
Сервер	Python 3.14 + FastAPI	Приложение учит Python и запускает код учеников. На той же платформе это делается штатными средствами
Работа с базой	SQLAlchemy 2.0	ORM: таблицы описаны классами, SQL руками не пишется
База	SQLite	Один файл. Доступ только через ORM, переезд на PostgreSQL меняет строку подключения
Страницы	Jinja2	HTML собирается на сервере и приходит готовым
Клиент	Ванильный JavaScript	Пять файлов, 417 строк. Ни React, ни prn, ни сборки
Пароли	bcrypt	Хранятся только хэши
Сессии	itsdangerous	Подписанная cookie, подделать нельзя
Нейросеть	OpenAI-совместимый шлюз	Генерация уроков и подсказок. Модель <code>deepseek-v4-flash</code>
Развёртывание	systemd + nginx	Свой сервер, HTTPS

Ванильный JavaScript значит «чистый, без фреймворка». Браузер получает файл и сразу исполняет. Ничего не нужно собирать перед выкладкой.

3. Как устроено

Запрос идёт сверху вниз, каждый слой знает только соседний.

```
браузер
  | HTTPS
nginx ———— отдаёт картинки и стили прямо с диска
  | остальное проксирует
FastAPI
  |
api/      — принимает запросы, отдаёт HTML или JSON
  |        знает про HTTP, не знает про бизнес-правила
services/ — бизнес-логика: монеты, серии, песочница, генерация уроков
  |        не знает, что существует HTTP
models/   — описание таблиц
  |
SQLite
```

Правило простое: `services` никогда не импортирует `FastAPI`, а `models` никогда не импортирует `services`. Поэтому логику можно проверять без поднятия сервера.

Отдельно в стороне стоят две вещи:

- **песочница** — отдельный процесс, в котором исполняется код ученика;
 - **шлюз нейросети** — внешний сервис, к которому ходят за уроками и подсказками.
-

4. Где что лежит

app/	
main.py	точка входа: собирает приложение, поднимает базу
config.py	все настройки из переменных окружения
database.py	подключение и сессии
models/	таблицы, 16 штук
user.py	пользователь, дни занятий, движение монет
learning.py	курс, модуль, урок, задание, решение, попытка теста
game.py	предметы, инвентарь, вращения колеса
reward.py	медали и мемы
admin.py	массовые начисления
api/	слой HTTP
pages.py	страницы: лендинг, тест, урок, магазин, профиль
actions.py	JSON API под префиксом /hog
admin.py	панель администратора
deps.py	общее для всех страниц: шаблонизатор, контекст
services/	бизнес-логика
economy.py	ЕДИНСТВЕННАЯ точка движения монет
sandbox.py	исполнение кода ученика
course.py	генерация уроков и их проверка
awards.py	выдача медалей, подбор мемов
auth.py	регистрация, вход, сессии
curriculum.py	каркас программы обучения
testbank.py	вопросы вводного теста
ai.py	клиент нейросети
i18n.py	переводы интерфейса
templates/	HTML-шаблоны
static/	css, js, картинки
docs/	вся документация и сборщики
deploy/	systemd и nginx
data/	файл базы (в репозиторий не попадает)
архив/	рабочие материалы, в репозиторий не попадают

5. Пять механизмов, которые стоит понимать

Исполнение кода ученика

Код не разбирается и не сравнивается с образцом. Он **запускается** в отдельном процессе с тремя ограничениями: не дольше пяти секунд, не больше 128 мегабайт памяти, не больше десяти тысяч символов вывода. Процесс стартует без доступа к переменным окружения и в пустом временном каталоге. Результат возвращается через служебную метку в выводе.

Перед запуском код проходит грубую проверку на запрещённые модули и вызовы. Это не полноценная изоляция, и в документации это признано прямо: для публичного сервиса нужен контейнер без сети.

Проверка сгенерированных уроков

Самая ценная идея в проекте. Нейросеть выдаёт урок вместе с эталонным решением и тестами. Прежде чем урок попадёт к ученику, **эталонное решение запускается в песочнице против этих же тестов**. Не сошлось — урок отбрасывается целиком и берётся шаблонный.

То есть враньё модели физически не доезжает до пользователя.

Экономика через одну точку

Любое изменение баланса проходит через один метод. Он записывает строку в журнал операций и запрещает уйти в минус. Поле с балансом существует только ради скорости чтения: настоящий баланс всегда можно пересчитать суммой по журналу и сверить.

Защита от двойных начислений

Двойной клик по колесу или две открытые вкладки не дают приз дважды. Работает не проверка в коде, а **уникальное ограничение в базе** на пару «пользователь и веха». Второй запрос упирается в него и отваливается. Проверку в коде можно обойти параллельным запросом, ограничение базы нельзя.

Так же защищены медали и задания внутри урока.

Работа без нейросети

Убери ключ из настроек, и приложение продолжит работать: уроки станут шаблонными, подсказки статичными, всё остальное не изменится. Внешний сервис не является точкой отказа.

6. Как запустить и выкатить

Локально:

```
./run.sh # поднимет окружение и запустит
```

Откроется на `http://127.0.0.1:8000/codehog/`. Подкаталог задаётся переменной `BASE_PATH` в файле `.env`.

На сервер:

```
rsync -az --delete --exclude '.venv' --exclude 'data' --exclude '.env*' \
  --exclude '.git' --exclude 'архив' ./ root@maks.my:/opt/codehog/
ssh root@maks.my 'chown -R www-data:www-data /opt/codehog && systemctl restart codehog'
```

Схема базы обновляется сама при старте, отдельных миграций запускать не нужно.

7. Куда смотреть при типовых задачах

Надо сделать	Файл
Изменить награды за урок или цены	<code>services/economy.py</code> , <code>services/seed.py</code>
Добавить тему в курс	<code>services/curriculum.py</code>
Изменить, что генерирует нейросеть	<code>services/course.py</code> — промпт и проверка
Добавить тип задания	<code>models/learning.py</code> , <code>services/course.py</code> , <code>api/actions.py</code> , <code>templates/lesson.html</code>
Добавить медаль	<code>services/awards.py</code> — каталог наверху файла
Заменить подборку мемов	таблица <code>memes</code> , код трогать не нужно
Добавить язык	<code>services/i18n.py</code>
Поменять внешний вид	<code>static/css/style.css</code> — токены в начале файла
Пересобрать схему базы для документации	<code>docs/собрать-схему.py</code>
Пересобрать документ в Word	<code>docs/собрать-документ.py</code>

8. Что честно не сделано

1. **Автотестов нет.** Проверки ручные и скриптами, воспроизводимые, но `pytest` это не заменяет.
2. **Макетов в Figma нет.** Дизайн сделан сразу в CSS по спецификации.
3. **Изоляция кода процессная, не контейнерная.** Для учебного стенда достаточно, для публичного сервиса мало.
4. **Раздела CMS нет.** Уроки и предметы задаются в коде, хотя медали и мемы уже вынесены в таблицы.
5. **Стек не согласован с преподавателем письменно.**